

# M2 - Sistemi operativi e linguaggi

Corso: CyberSecurity

Studente: Martina Foresta

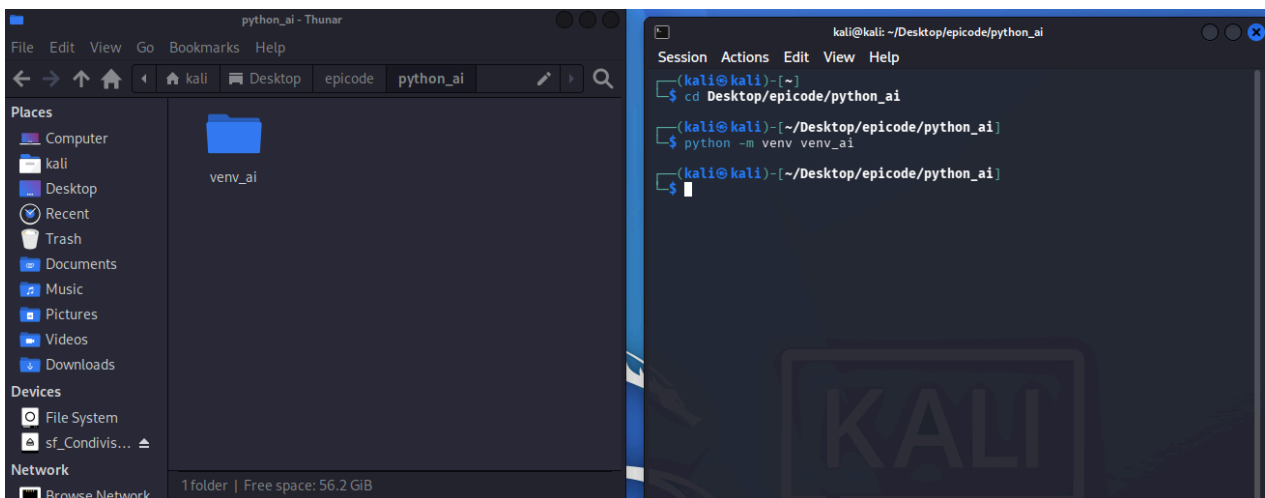
Esercizio: Python & AI per la Cyber Security

Traccia: configurare l'ambiente Python per Gemini, creare uno script Hello Gemini e realizzare un chatbot Security Assistant che mantenga una conversazione interattiva.

Per iniziare mi sono spostata nella cartella `~/Desktop/epicode/python_ai`,

Come prima cosa ho creato l'ambiente virtuale con il comando `python -m venv venv_ai`.

Questo ambiente mantiene separate le librerie del progetto da quelle già presenti in Kali Linux.



La cartella `venv_ai` visibile nella directory conferma che la creazione dell'ambiente e' avvenuta correttamente.

Successivamente ho attivato l'ambiente virtuale con `source venv_ai/bin/activate`. La scritta `(venv_ai)` davanti al prompt indica che Python e pip stanno lavorando all'interno dell'ambiente appena creato.

```
(kali@kali)-[~/Desktop/epicode/python_ai]
$ source venv_ai/bin/activate

(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ pip install google-genai pandas numpy
```

Ho installato le dipendenze necessarie con `pip install google-genai pandas numpy`. La libreria `google-genai` consente agli script Python di comunicare con Gemini; `pandas` e `numpy` rimangono disponibili per eventuali elaborazioni di dati.

```
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ pip install google-genai pandas numpy
Collecting google-genai
  Downloading google_genai-2.20.0-py3-none-any.whl.metadata (56 kB)
Collecting pandas
  Downloading pandas-3.0.5-cp314-cp314-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting numpy
  Downloading numpy-2.5.2-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting anyio<5.0.0, ≥4.8.0 (from google-genai)
  Downloading anyio-4.14.2-py3-none-any.whl.metadata (4.6 kB)
Collecting google-auth<3.0.0, ≥2.56.0 (from google-genai)
  Downloading google_auth-2.57.0-py3-none-any.whl.metadata (6.0 kB)
Collecting httpx<1.0.0, ≥0.28.1 (from google-genai)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pydantic<3.0.0, ≥2.12.5 (from google-genai)
  Downloading pydantic-2.19.0-py3-none-any.whl.metadata (110 kB)
Collecting requests<3.0.0, ≥2.28.1 (from google-genai)
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting tenacity<9.2.0, ≥8.2.3 (from google-genai)
  Downloading tenacity-9.1.4-py3-none-any.whl.metadata (1.2 kB)
Collecting websockets<17.0, ≥13.0.0 (from google-genai)
  Downloading websockets-16.1.1-cp314-cp314-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)
Collecting typing_extensions<5.0.0, ≥4.15.0 (from google-genai)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting distro<2.1.0, ≥1.7.0 (from google-genai)
  Downloading distro-1.9.0-py3-none-any.whl.metadata (6.8 kB)
Collecting sniffio (from google-genai)
  Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting idna<3.0, ≥2.8 (from anyio<5.0.0, ≥4.8.0->google-genai)
  Downloading idna-3.10-py3-none-any.whl.metadata (9.2 kB)
Collecting pyasn1-modules<0.4.2, ≥0.2.1 (from google-auth<3.0.0, ≥2.56.0->google-genai)
  Downloading pyasn1_modules-0.4.2-py3-none-any.whl.metadata (3.5 kB)
Collecting cryptography<41.0.5, ≥41.0.5 (from google-auth<3.0.0, ≥2.56.0->google-genai)
  Downloading cryptography-40.0.1-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting certifi<2026.0.0, ≥2025.12.31 (from httpx<1.0.0, ≥0.28.1->google-genai)
  Downloading certifi-2026.0.2-py3-none-any.whl.metadata (2.3 kB)
Collecting httpcore<1.0, ≥1.0.0 (from httpx<1.0.0, ≥0.28.1->google-genai)
  Downloading httpcore-1.0.9-py3-none-any.whl.metadata (21 kB)
Collecting h11<0.16.0, ≥0.16.0 (from httpcore<1.0, ≥1.0.0->httpx<1.0.0, ≥0.28.1->google-genai)
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting annotated_types<0.7.0, ≥0.6.0 (from pydantic<3.0.0, ≥2.12.5->google-genai)
  Downloading annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic_core<2.46.5, ≥2.46.5 (from pydantic<3.0.0, ≥2.12.5->google-genai)
  Downloading pydantic_core-2.46.5-cp314-cp314-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.6 kB)
Collecting typing_inspection<0.4.2, ≥0.4.2 (from pydantic<3.0.0, ≥2.12.5->google-genai)
  Downloading typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Collecting charset_normalizer<4, ≥2 (from requests<3.0.0, ≥2.28.1->google-genai)
  Downloading charset_normalizer-3.5.1-cp314-cp314-manylinux2014_x86_64.manylinux_2_28_x86_64.whl.metadata (45 kB)
Collecting urllib3<3, ≥1.26 (from requests<3.0.0, ≥2.28.1->google-genai)
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)
```

Al termine dell'installazione, la scritta `Successfully installed` conferma che i pacchetti e le relative dipendenze sono stati installati correttamente nel virtual environment.

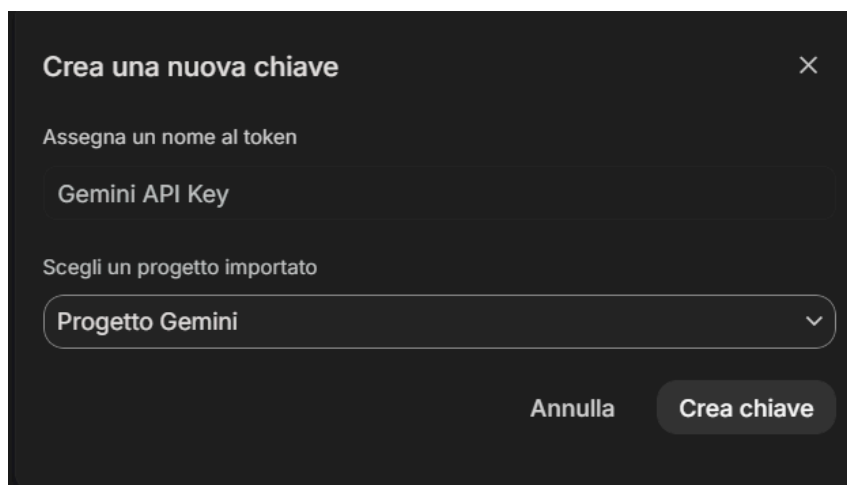
```
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ pip freeze >> requirements.txt

(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ ls
requirements.txt  venv_ai

(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$
```

Infine ho eseguito `pip freeze >> requirements.txt` per salvare l'elenco delle librerie installate. Il file `requirements.txt` permette di ricreare lo stesso ambiente in futuro.

*Per collegare Gemini agli script ho creato una API key in Google AI Studio, associandola al progetto Gemini.*



*Ho quindi salvato la chiave nella variabile d'ambiente GEMINI\_API\_KEY. Con il comando `export GEMINI_API_KEY=...` la chiave è disponibile nella sessione corrente; aggiungendo il comando al file `~/.zshrc` rimane disponibile anche dopo aver riaperto il terminale.*

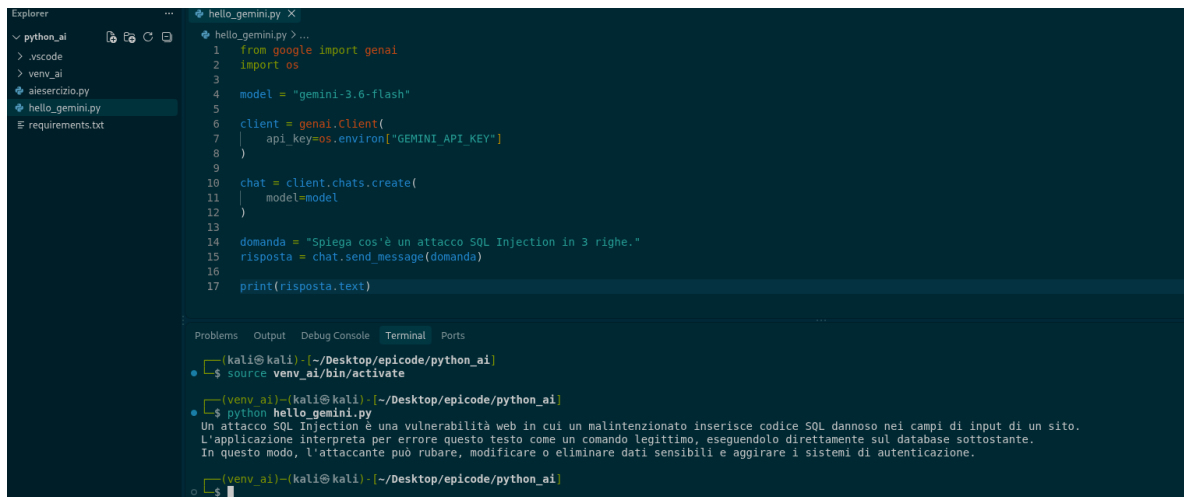
```
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ export GEMINI_API_KEY=[REDACTED]

(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ echo export GEMINI_API_KEY=\"${GEMINI_API_KEY}\" >> ~/.zshrc
```

*In questo modo gli script possono leggere la chiave con `os.environ["GEMINI_API_KEY"]`, mantenendo il dato sensibile separato dai file sorgente.*

Dopo il setup ho creato il file `hello_gemini.py`. Lo script importa `google.genai` e `os`, imposta il modello `gemini-3.6-flash` e crea un client usando la chiave presente nella variabile d'ambiente.

Con `client.chats.create` viene aperta una chat e il metodo `chat.send_message` invia la domanda: *Spiega cos'è un attacco SQL Injection in 3 righe*. Il testo della risposta viene infine mostrato con `print(risposta.text)`.



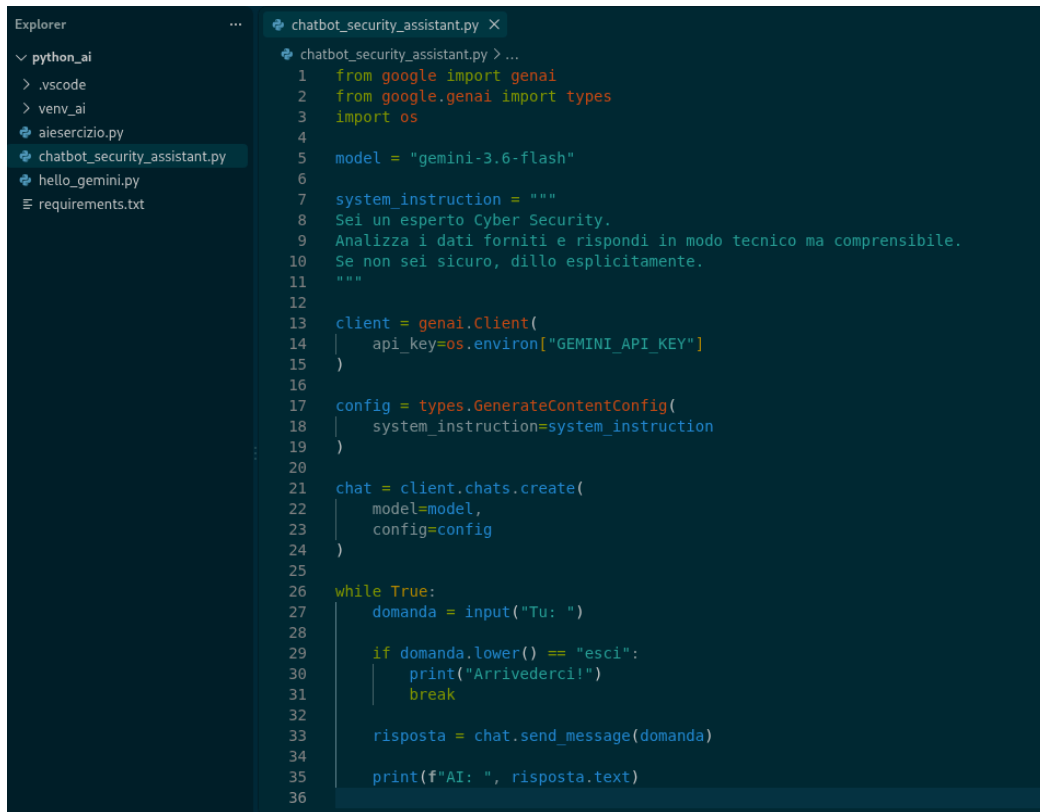
```
Explorer
python_ai
  .vscode
  venv_ai
  aiesercizio.py
  hello_gemini.py
  requirements.txt

hello_gemini.py X
1 from google import genai
2 import os
3
4 model = "gemini-3.6-flash"
5
6 client = genai.Client(
7     api_key=os.environ["GEMINI_API_KEY"]
8 )
9
10 chat = client.chats.create(
11     model=model
12 )
13
14 domanda = "Spiega cos'è un attacco SQL Injection in 3 righe."
15 risposta = chat.send_message(domanda)
16
17 print(risposta.text)

Problems Output Debug Console Terminal Ports
(kali@kali)-[~/Desktop/epicode/python_ai]
$ source venv_ai/bin/activate
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
$ python hello_gemini.py
Un attacco SQL Injection è una vulnerabilità web in cui un malintenzionato inserisce codice SQL dannoso nei campi di input di un sito. L'applicazione interpreta per errore questo testo come un comando legittimo, eseguendolo direttamente sul database sottostante. In questo modo, l'attaccante può rubare, modificare o eliminare dati sensibili e aggirare i sistemi di autenticazione.
(venv_ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
```

L'esecuzione con `python hello_gemini.py` ha prodotto una spiegazione sintetica: Gemini descrive la SQL Injection come una vulnerabilità che sfrutta input non gestiti correttamente per eseguire comandi sul database. Il prompt `(venv_ai)` ricompare al termine, confermando che lo script è stato completato.

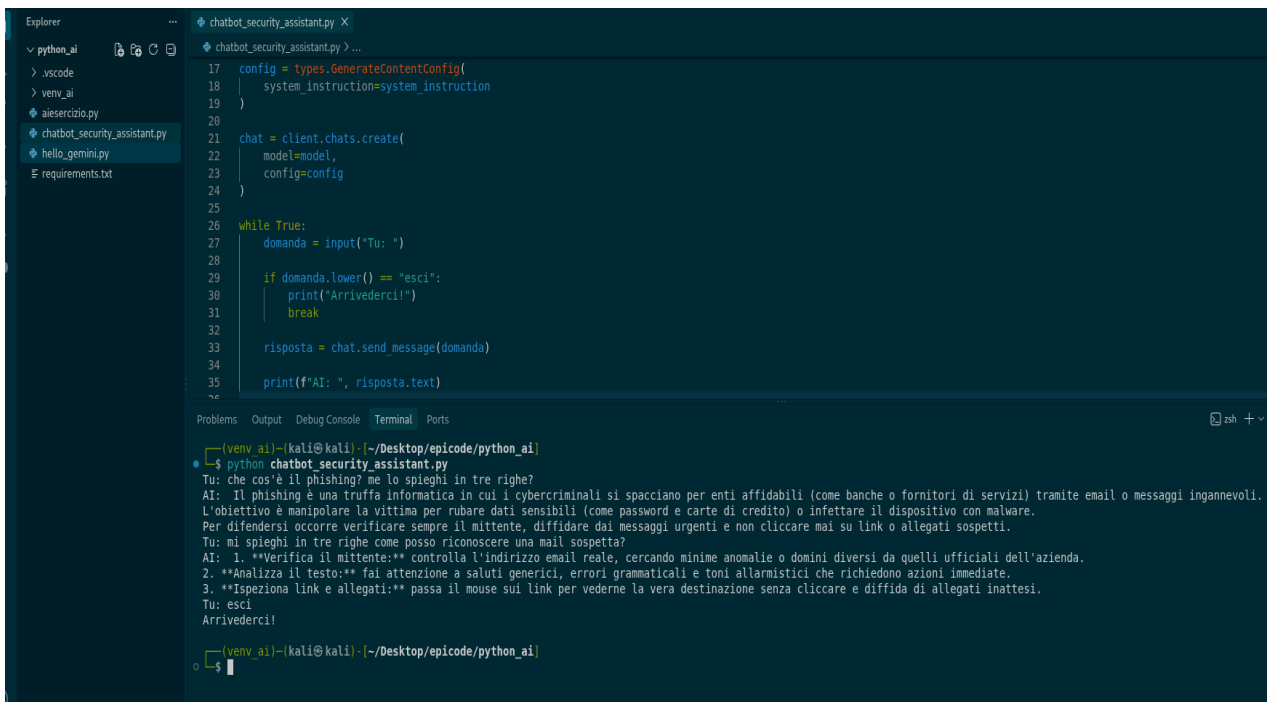
Nella seconda parte ho creato `chatbot_security_assistant.py`, un chatbot interattivo dedicato alla cybersecurity. Oltre agli import, nel programma è presente una system instruction che assegna a Gemini il ruolo di esperto Cyber Security e richiede risposte tecniche ma comprensibili.



```
1 from google import genai
2 from google.genai import types
3 import os
4
5 model = "gemini-3.6-flash"
6
7 system_instruction = """
8 Sei un esperto Cyber Security.
9 Analizza i dati forniti e rispondi in modo tecnico ma comprensibile.
10 Se non sei sicuro, dillo esplicitamente.
11 """
12
13 client = genai.Client(
14     api_key=os.environ["GEMINI_API_KEY"]
15 )
16
17 config = types.GenerateContentConfig(
18     system_instruction=system_instruction
19 )
20
21 chat = client.chats.create(
22     model=model,
23     config=config
24 )
25
26 while True:
27     domanda = input("Tu: ")
28
29     if domanda.lower() == "esci":
30         print("Arrivederci!")
31         break
32
33     risposta = chat.send_message(domanda)
34
35     print(f"AI: ", risposta.text)
36
```

La configurazione `GenerateContentConfig` passa la system instruction alla chat. Il ciclo `while True` permette poi di inserire più domande una dopo l'altra. Se l'utente digita `esci`, il programma stampa `Arrivederci!` e termina il ciclo.

Ho avviato il chatbot dal terminale con `python chatbot_security_assistant.py`, sempre con l'ambiente virtuale attivo. Durante la prova ho chiesto una spiegazione del phishing e come riconoscere una mail sospetta.



The screenshot shows a VS Code editor with a file explorer on the left containing files like `python_ai`, `venv_ai`, `aiesercizio.py`, `chatbot_security_assistant.py`, `hello_gemini.py`, and `requirements.txt`. The main editor displays the `chatbot_security_assistant.py` script, which uses the Google Gemini API to generate responses. The terminal at the bottom shows the execution of the script in a virtual environment, with a user asking about phishing and the chatbot providing a detailed explanation and tips for recognizing suspicious emails. The user then types 'esci' to exit the chat, and the chatbot responds with 'Arrivederci!'.

```
17 config = types.GenerateContentConfig(
18     system_instruction=system_instruction
19 )
20
21 chat = client.chats.create(
22     model=model,
23     config=config
24 )
25
26 while True:
27     domanda = input("Tu: ")
28
29     if domanda.lower() == "esci":
30         print("Arrivederci!")
31         break
32
33     risposta = chat.send_message(domanda)
34
35     print(f"AI: ", risposta.text)
```

Problems Output Debug Console Terminal Ports

```
(venv ai)-(kali@kali)-[~/Desktop/epicode/python_ai]
└─$ python chatbot_security_assistant.py
Tu: che cos'è il phishing? me lo spieghi in tre righe?
AI: Il phishing è una truffa informatica in cui i cybercriminali si spacciano per enti affidabili (come banche o fornitori di servizi) tramite email o messaggi ingannevoli.
L'obiettivo è manipolare la vittima per rubare dati sensibili (come password e carte di credito) o infettare il dispositivo con malware.
Per difendersi occorre verificare sempre il mittente, diffidare dai messaggi urgenti e non cliccare mai su link o allegati sospetti.
Tu: mi spieghi in tre righe come posso riconoscere una mail sospetta?
AI: 1. **Verifica il mittente:** controlla l'indirizzo email reale, cercando minime anomalie o domini diversi da quelli ufficiali dell'azienda.
2. **Analizza il testo:** fai attenzione a saluti generici, errori grammaticali e toni allarmistici che richiedono azioni immediate.
3. **Ispeziona link e allegati:** passa il mouse sui link per vederne la vera destinazione senza cliccare e diffida di allegati inattesi.
Tu: esci
Arrivederci!
```

Il chatbot ha spiegato che il phishing usa messaggi ingannevoli per ottenere dati sensibili. Alla seconda domanda ha dato indicazioni utili: verificare il mittente, osservare saluti generici o errori e non aprire link o allegati inattesi. Scrivendo `esci`, la chat si è chiusa correttamente. In conclusione, l'esercizio è stato svolto correttamente: ho configurato un ambiente Python isolato, protetto la API key tramite variabile d'ambiente, realizzato uno script Hello Gemini e creato un chatbot Security Assistant capace di mantenere una conversazione interattiva su temi di cybersecurity.